

Tuples

Learning Objectives

- Define tuples and explain how they differ from lists in mutability and use cases
- Use tuples for function returns, dictionary keys, and data protection
- Apply tuple unpacking and understand named tuples for cleaner code

Engage (5-7 minutes)

A list is like a whiteboard—you can erase and rewrite anything. A tuple is like ink on paper—once written, it cannot be changed. This immutability is not a limitation; it is a feature that guarantees data integrity. Tuples are used for coordinates, database records, function returns with multiple values, and anywhere data should not be modified. Today we learn when to choose tuples over lists.

Explore (10-12 minutes)

Create a tuple representing a point in 2D space: (3, 4). Try to change the first element—observe the error. Now create a tuple of five subjects and access them using negative indices. Create a dictionary with tuples as keys: {(1, 2): "point_a", (3, 4): "point_b"}. Can you do the same with lists as keys? Experiment and explain the difference.

Explain (10-12 minutes)

Tuples are immutable ordered collections, meaning once created, their elements cannot be added, removed, or changed. They use parentheses (1, 2, 3) or just commas 1, 2, 3. Tuples support indexing, slicing, and iteration like lists, but not modification methods. Because they are immutable, tuples can be used as dictionary keys and set elements—lists cannot. Tuple unpacking allows assigning multiple variables at once: `x, y, z = (1, 2, 3)`. Tuples are faster than lists and use less memory.

Elaborate (8-10 minutes)

Implement a function that returns multiple values as a tuple: calculate mean, median, and mode of a list and return all three. Use tuple unpacking to capture the results. Create a list of tuples representing (student_name, grade) pairs and sort by grade. Implement a function that swaps two variables without a temporary variable using tuple unpacking. Explore named tuples from the collections module and create a Point named tuple. Discuss: why are tuples preferred for function returns that should not be modified?

Evaluate (5-8 minutes)

Write the difference between a list and a tuple in three key areas: mutability, use as dictionary keys, and performance. Given `data = (10, 20, 30, 40, 50)`, write the output of: `data[2]`, `data[-1]`, `data[1:4]`, and `len(data)`. Then write a function that uses tuple unpacking to swap the first and last elements of a list.